Sistemas Digitales — Semana 3

Temporizadores del ESP32

El temporizador (timer) es un circuito del microcontrolador que cuenta pulsos de reloj. Sirve para medir tiempos o dar un aviso cuando se cumple un intervalo. No es una variable ni una función del programa: es un periférico de hardware.

La CPU configura sus registros, lo arranca y puede detenerlo. Mientras tanto el contador avanza solo, sin que la CPU tenga que sumarle 1 en cada vuelta de loop().

delay(), millis() y timer

- delay(): detiene el avance normal del código durante la espera.
- millis(): permite revisar cuánto tiempo pasó, pero depende de que el programa vuelva a comprobarlo.
- Timer: el hardware cuenta por su lado y puede generar una interrupción al llegar al valor configurado.

Reloj, prescaler y tick

En el ESP32 clásico se usa como referencia un reloj de periféricos de 80 MHz. Son 80 millones de pulsos cada segundo. El prescaler divide esa frecuencia para que el contador avance más despacio.

Ejemplo: 80 000 000 / 80 = 1 000 000 ticks por segundo. Entonces cada tick dura 1 microsegundo (1 µs). A esto se le llama resolución del timer.

- Frecuencia de ticks = frecuencia del reloj / prescaler.
- Resolución = 1 / frecuencia de ticks.

Alarma y tiempo

La alarma es el número de ticks al que queremos llegar. Al alcanzarlo se genera el evento que puede avisar a la CPU.

- Tiempo = alarma × resolución.
- Alarma = tiempo × frecuencia de ticks.

Si queremos un aviso a los 500 ms con 1 MHz: 0,5 × 1 000 000 = 500 000. Ese es el valor de alarma. Para 10 ms sería 10 000.

Con autorrecarga, al llegar a la alarma el contador vuelve a empezar y el aviso se repite. No es igual al desbordamiento, que ocurre cuando se supera el valor máximo del contador.

Sistemas Digitales — Semana 3

Temporizadores que tiene el ESP32

El ESP32 clásico tiene 4 timers de propósito general, repartidos en dos grupos. Sus contadores son de 64 bits. En comparación, el ATmega328P usa temporizadores de 8 y 16 bits.

Se pueden configurar para trabajar de forma independiente. Lo importante no es memorizar todos los registros, sino saber cómo se relacionan el reloj, el prescaler, el contador y la alarma.

La API y la versión del core

La API son funciones ya preparadas para controlar el hardware sin escribir directamente todos los registros. El core de ESP32 es el paquete que permite programar la placa desde Arduino IDE.

Hay que revisar la versión del core: en la 3.x cambiaron varias funciones de temporizadores respecto a la 2.x. Por eso no siempre funciona un código copiado de un tutorial antiguo.

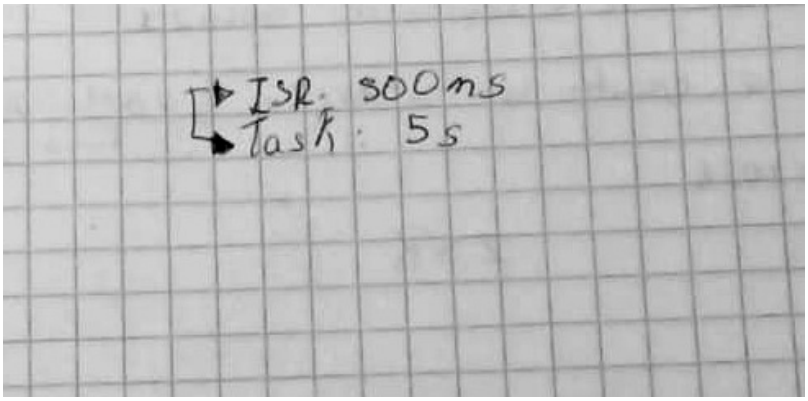
Cuando suena la alarma: interrupción

El timer puede enviar una interrupción a la CPU. Entonces se ejecuta una función especial llamada ISR (rutina de servicio de interrupción) y después el programa sigue con lo que estaba haciendo.

- Callback: función que se registra para ejecutarse al producirse el evento.
- IRAM_ATTR: se usa para ubicar el código de la ISR en memoria interna de instrucciones, cuando corresponde.
- volatile: indica que una variable puede cambiar inesperadamente, por ejemplo desde una ISR. No hace que las operaciones sean atómicas.

La ISR tiene que ser corta

En los apuntes aparece la idea de que la ISR actúe rápido, mientras la tarea normal puede tardar más. Por eso suele usarse una bandera: la ISR la cambia y loop() realiza el trabajo más largo, por ejemplo mostrar un mensaje.



Ejemplo de funcionamiento: el temporizador avisa cada 500 ms; la ISR marca que ocurrió el evento y el programa principal decide qué hacer. No conviene poner esperas largas dentro de la ISR.

También se puede trabajar con interrupciones externas por cambios en un pin, como al presionar un botón. Eso se estudia con mayor detalle en la siguiente semana.